

计算机系统基础 PA1

姓名： 刘宇泽

学号： 2411334

概述

实验目的

- 熟悉 GNU/Linux 平台
- 初步探究“程序在计算机上运行”的相关原理
- 初步学习 GDB 并在 PA 上实现简易调试器

实验内容

- 阶段一：模拟寄存器结构，实现调试器基本功能。
- 阶段二：实现调试功能的表达式求值，并完善阶段一中的扫描内存函数。
- 阶段三：实现调试功能中的监视点，学习断点相关知识与 i386 手册。

阶段一

nemu执行流程

阅读nemu/src/main.c,NEMU首先通过 `init_monitor(argc,argv)` 完成相关初始化工作

```

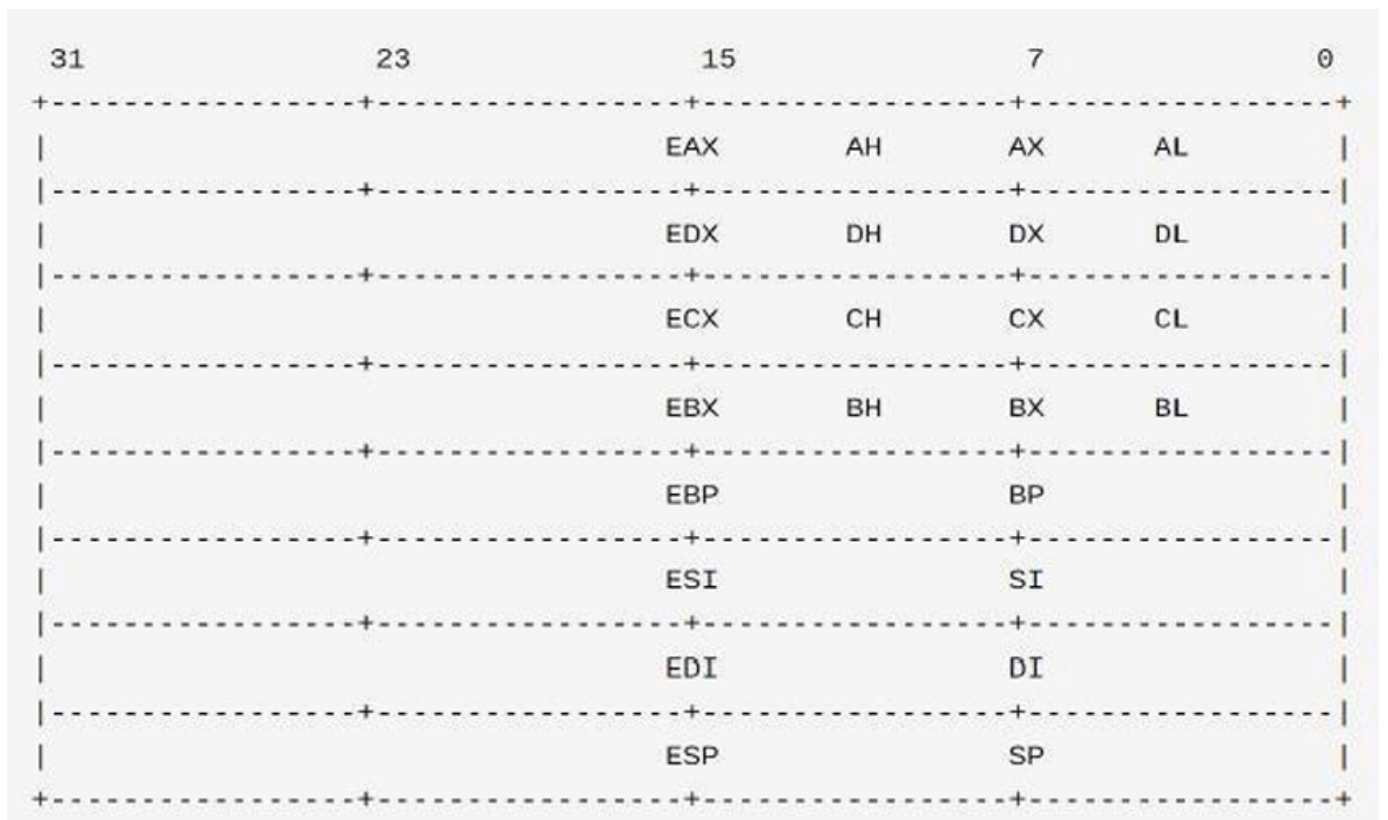
int init_monitor(int argc, char *argv[]) {
    /* 执行一些全局初始化操作 */
    /* 解析命令行参数 */
    parse_args(argc, argv);
    /* 打开日志文件 */
    init_log();
    /* 测试 `CPU_state' 结构体的实现 */
    reg_test();
#ifdef DIFF_TEST
    /* 创建子进程以执行差分测试 */
    init_difftest();
#endif
    /* 将镜像加载到内存 */
    load_img();
    /* 初始化虚拟计算机系统 */
    restart();
    /* 编译正则表达式 */
    init_regex();
    /* 初始化监视点池 */
    init_wp_pool();
    /* 初始化设备 */
    init_device();
    /* 显示欢迎信息 */
    welcome();
    return is_batch_mode;
}

```

实现正确寄存器结构体

结构体与函数介绍

- 寄存器结构：除程序计数器eip外，32位、16位、8位寄存器各8个



- Union 结构: Union中的各变量互斥，共享同一内存首地址，寄存器的结构实现就是基于该特性实现的。
- reg_test() 函数：寄存器测试函数

```

void reg_test() {
    srand(time(0));
    uint32_t sample[8];
    uint32_t eip_sample = rand();
    cpu.eip = eip_sample;

    int i;
    for (i = R_EAX; i <= R_EDI; i++) {
        sample[i] = rand();
        reg_l(i) = sample[i];
        assert(reg_w(i) == (sample[i] & 0xffff));
    }

    assert(reg_b(R_AL) == (sample[R_EAX] & 0xff));
    assert(reg_b(R_AH) == ((sample[R_EAX] >> 8) & 0xff));
    assert(reg_b(R_BL) == (sample[R_EBX] & 0xff));
    assert(reg_b(R_BH) == ((sample[R_EBX] >> 8) & 0xff));
    assert(reg_b(R_CL) == (sample[R_ECX] & 0xff));
    assert(reg_b(R_CH) == ((sample[R_ECX] >> 8) & 0xff));
    assert(reg_b(R_DL) == (sample[R_EDX] & 0xff));
    assert(reg_b(R_DH) == ((sample[R_EDX] >> 8) & 0xff));

    assert(sample[R_EAX] == cpu.eax);
    assert(sample[R_ECX] == cpu.ecx);
    assert(sample[R_EDX] == cpu.edx);
    assert(sample[R_EBX] == cpu.ebx);
    assert(sample[R_ESP] == cpu.esp);
    assert(sample[R_EBP] == cpu.ebp);
    assert(sample[R_ESI] == cpu.esi);
    assert(sample[R_EDI] == cpu.edi);

    assert(eip_sample == cpu.eip);
}

```

该测试函数主要为两个部分：

- 检验 32、16、8 位寄存器满足图片所示的寄存器结构。
- 确保可直接通过寄存器名访问 32 位寄存器的内容。

代码实现

- 修改nemu/include/cpu/reg.h

```

#ifndef __REG_H__
#define __REG_H__
#include "common.h"
enum { R_EAX, R_ECX, R_EDX, R_EBX, R_ESP, R_EBP, R_ESI, R_EDI };
enum { R_AX, R_CX, R_DX, R_BX, R_SP, R_BP, R_SI, R_DI };
enum { R_AL, R_CL, R_DL, R_BL, R_AH, R_CH, R_DH, R_BH };
typedef struct {
    union {
        union {
            uint32_t _32;
            uint16_t _16;
            uint8_t _8[2];
        } gpr[8];
        /* Do NOT change the order of the GPRs' definitions. */
        /* 与gpr[0]-gpr[7]一一对应，共享同一块内存空间 */
        struct {
            rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
        };
    };
    vaddr_t eip;
} CPU_state;
extern CPU_state cpu;
static inline int check_reg_index(int index) {
    assert(index >= 0 && index < 8);
    return index;
}
#define reg_l(index) (cpu.gpr[check_reg_index(index)]._32)
#define reg_w(index) (cpu.gpr[check_reg_index(index)]._16)
#define reg_b(index) (cpu.gpr[check_reg_index(index) & 0x3]._8[index >> 2])
extern const char* regsl[];
extern const char* regsw[];
extern const char* regsb[];
static inline const char* reg_name(int index, int width) {
    assert(index >= 0 && index < 8);
    switch (width) {
        case 4: return regsl[index];
        case 1: return regsb[index];
        case 2: return regsw[index];
        default: assert(0);
    }
}
}
#endif

```

运行结果

输入 make run, 成功输出 welcome

```
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the
default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 15:32:00, Apr 15 2026
For help, type "help"
(nemu) |
```

问题：究竟要执行多久

打开nemu/src/monitor/cpu-exec.c, 查看 `cpu_exec()` 函数的实现, 发现函数模仿CPU的工作方式, 不断执行n条指令, 直到指令执行完毕或进入nemu_trap, 才退出指令执行的循环。

```
/* Simulate how the CPU works. */
void cpu_exec(uint64_t n) {
    if (nemu_state == NEMU_END) {
        printf("Program execution has ended. To restart the program, exit NEMU and run again.\n");
        return;
    }
    nemu_state = NEMU_RUNNING;

    bool print_flag = n < MAX_INSTR_TO_PRINT;

    for (; n > 0; n --) {
        /* Execute one instruction, including instruction fetch,
         * instruction decode, and the actual execution. */
        exec_wrapper(print_flag);

#ifdef DEBUG
        /* TODO: check watchpoints here. */

#endif

#ifdef HAS_IOE
        extern void device_update();
        device_update();
#endif

        if (nemu_state != NEMU_RUNNING) { return; }
    }

    if (nemu_state == NEMU_RUNNING) { nemu_state = NEMU_STOP; }
}
```

所以传入-1即uint64_t中的 $2^{64}-1$ ，就算 CPU 每秒执行 10 亿条指令，跑满这个数也需要 约 5800 亿年（远超宇宙年龄），对我们的程序来说，等价于「无限执行」。

所以当你输入 c (continue) 命令时，NEMU 会给 cpu_exec 传 -1，让程序一直跑，直到遇到 nemu_trap 指令，才会退出执行循环。

```
[src/monitor/monitor.c,30,welcome] Build time: 15:32:00, Apr 15 2026
For help, type "help"
(nemu) c
nemu: HIT GOOD TRAP at eip = 0x00100026
```

问题：谁来指示程序的结束

实验一：写一个普通的C程序（验证main返回会退出）

```
#include <stdio.h>
int main() {
    printf("Hello from main()\n");
    return 0; // main 返回
}
```

```
liuyuze@liuyuze-virtual-machine:~/ics2017/nemu$ gcc test1.c -o test1
liuyuze@liuyuze-virtual-machine:~/ics2017/nemu$ ./test1
Hello from main()
```

现象：程序打印 Hello from main() 后正常退出，符合课上认知。

实验二：核main() 不返回，程序也能退出（exit() 才是关键）

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    printf("Hello from main()\n");
    exit(0);
    printf("This line will never print\n"); // 死代码
}
```

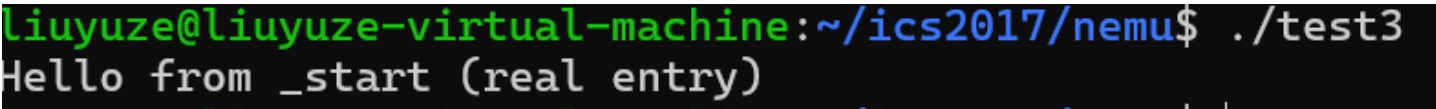
```
liuyuze@liuyuze-virtual-machine:~/ics2017/nemu$ gcc test2.c -o test2
liuyuze@liuyuze-virtual-machine:~/ics2017/nemu$ ./test2
Hello from main()
```

现象：

- 只打印 Hello from main()，第二行永远不会执行
- 程序正常退出，说明 **main() 不需要返回，程序也能结束 **，exit() 才是真正的退出触发点

实验三：跳过 main()，直接看程序入口

```
#include <stdio.h>
#include <stdlib.h>
// 自定义入口，替代运行时库的 _start
void _start() {
    printf("Hello from _start (real entry)\n");
    exit(0); // 直接退出，完全不经过 main()
}
```



现象：

- 程序正常打印 Hello from _start (real entry) 并退出
- 完全没有 main() 函数，证明 main() 不是程序的必须入口，只是运行时库的一个调用目标

基础设施：简易调试器

命令	格式	使用举例	说明
帮助(1)	help	help	打印命令的帮助信息
继续运行(1)	c	c	继续运行被暂停的程序
退出(1)	q	q	退出NEMU
单步执行	si [N]	si 10	让程序单步执行 N 条指令后暂停执行，当 N 没有给出时，缺省为 1
打印程序状态	info SUBCMD	info r info w	打印寄存器状态 打印监视点信息
表达式求值	p EXPR	p \$eax + 1	求出表达式 EXPR 的值，EXPR 支持的运算请见调试中的表达式求值小节
扫描内存(2)	x N EXPR	x 10 \$esp	求出表达式 EXPR 的值，将结果作为起始内存地址，以十六进制形式输出连续的 N 个4字节
设置监视点	w EXPR	w *0x2000	当表达式 EXPR 的值发生变化时，暂停程序执行
删除监视点	d N	d 2	删除序号为 N 的监视点

代码实现

修改nemu/src/monitor/debug/ui.c

单步执行

- `cpu_exec(uint64_t n)`函数模拟 CPU 的工作方式，执行n条指令，单步执行命令只需简单调用该函数即可

```
static int cmd_si(char *args) {
    uint64_t N=0;
    if(args==NULL)
        N=1;
    else{
        int nRet=sscanf(args,"%llu",&N);
        if(nRet<=0){
            printf("args error in cmd_si\n");
            return 0;
        }
    }
    cpu_exec(N);
    return 0;
}
```

测试

```
[src/monitor/monitor.c,30,welcome] Build time: 15:32:00, Apr 15 2026
For help, type "help"
(nemu) si 5
100000:  b8 34 12 00 00      movl $0x1234,%eax
100005:  b9 27 00 10 00      movl $0x100027,%ecx
10000a:  89 01               movl %eax,(%ecx)
10000c:  66 c7 41 04 01 00   movw $0x1,0x4(%ecx)
100012:  bb 02 00 00 00      movl $0x2,%ebx
(nemu) si 6
100017:  66 c7 84 99 00 e0 ff ff 01 00   movw $0x1,-0x2000(%ecx,%ebx,4)
100021:  b8 00 00 00 00      movl $0x0,%eax
nemu: HIT GOOD TRAP at eip = 0x00100026
100026:  d6                  nemu trap (eax = 0)
```

打印寄存器

- 通过 `regsl[i]` / `regsw[i]` / `regsb[i]` 调用 `reg_l(i)` / `reg_w(i)` / `reg_b(i)` 函数实现寄存器的读取
取"args error in cmd_info\n"

```

static int cmd_info(char *args) {
    char s;
    if(args == NULL){
        printf("args error in cmd_info\n");
        return 0;
    }
    int nRet = sscanf(args, "%c", &s);
    if(nRet <= 0){
        printf("args error in cmd_info\n");
        return 0;
    }
    if(s == 'r'){
        int i;
        for(i=0; i<8; i++)
            printf("%s\t0x%x\n", regsl[i], reg_l(i));
        printf("eip\t0x%x\n", cpu.eip);
        for(i=0; i<8; i++)
            printf("%s\t0x%x\n", regsw[i], reg_w(i));
        for(i=0; i<8; i++)
            printf("%s\t0x%x\n", regsb[i], reg_b(i));
        return 0;
    }
    printf("args error in cmd_info\n");
    return 0;
}

```

测试

```
(nemu) info r
eax      0x0
ecx      0x100027
edx      0x78d7ee3b
ebx      0x2
esp      0x761c28d6
ebp      0x536732
esi      0x42f7fcd3
edi      0x2e1e0984
eip      0x100027
ax       0x0
cx       0x27
dx       0xee3b
bx       0x2
sp       0x28d6
bp       0x6732
si       0xfcd3
di       0x984
al       0x0
cl       0x27
dl       0x3b
bl       0x2
ah       0x0
ch       0x0
dh       0xee
bh       0x0
```

扫描内存

- 通过 `vaddr_read` 函数 读取 4 字节内存数据，将 32 位数据拆分为 4 个字节，按「地址 + 32 位值 + 逐字节十六进制」的格式打印，每轮循环后地址自增 4 字节，最终完成指定范围内内存的可视化输出。

```

static int cmd_x(char *args) {
    if(!args){
        printf("args error in cmd_si\n");
        return 0;
    }
    char* args_end= args + strlen(args),*first_args=strtok(args," ");
    if(!first_args){
        printf("args error in cmd_si\n");
        return 0;
    }
    char *exprs=first_args+strlen(first_args)+1;
    if(exprs>=args_end){
        printf("args error in cmd_si\n");
        return 0;
    }
    int n=atoi(first_args);
    bool success;
    vaddr_t addr=expr(exprs,&success);
    if(success==false)
        printf("error in expr()\n");
    printf("Memory:");
    printf("\n");
    for(int i=0;i<n;i++){
        printf("0x%x:",addr);
        uint32_t val=vaddr_read(addr,4);
        uint8_t *by =(uint8_t *)&val;
        printf("0x");
        for(int j=3;j>=0;j--){
            printf("%02x",by[j]);
        }
        printf("\n");
        addr+=4;
    }
    return 0;
}

```

测试

```

liuyuze@liuyuze-virtual-machine:~/ics2017/nemu$ make run
./build/nemu -l ./build/nemu-log.txt
[src/monitor/monitor.c,47,load_default_img] No image is given. Use the
default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 18:32:17, Apr 15 2026
For help, type "help"
(nemu) x 10 0x100000
Memory:
0x100000:0x001234b8
0x100004:0x0027b900
0x100008:0x01890010
0x10000c:0x0441c766
0x100010:0x02bb0001
0x100014:0x66000000
0x100018:0x009984c7
0x10001c:0x01ffffe0
0x100020:0x0000b800
0x100024:0x00d60000

```

```

static inline int load_default_img() {
    const uint8_t img [] = {
        0xb8, 0x34, 0x12, 0x00, 0x00,           // 100000: movl $0x1234,%eax
        0xb9, 0x27, 0x00, 0x10, 0x00,           // 100005: movl $0x100027,%ecx
        0x89, 0x01,                             // 10000a: movl %eax,(%ecx)
        0x66, 0xc7, 0x41, 0x04, 0x01, 0x00,     // 10000c: movw $0x1,0x4(%ecx)
        0xbb, 0x02, 0x00, 0x00, 0x00,           // 100012: movl $0x2,%ebx
        0x66, 0xc7, 0x84, 0x99, 0x00, 0xe0,     // 100017: movw $0x1,-0x2000(%ecx,%ebx,4)
        0xff, 0xff, 0x01, 0x00,
        0xb8, 0x00, 0x00, 0x00, 0x00,           // 100021: movl $0x0,%eax
        0xd6,                                   // 100026: nemu_trap
    };
}

```

镜像对比验证正确

阶段二

实现算术表达式词法分析

相关变量与函数

1. 枚举：Token 类型编码

```
enum {
    TK_NOTYPE = 256,
    TK_EQ
};
```

- 作用：为所有 Token 分配唯一标识 ID
- 基础 ASCII 字符 (+) 直接用 ASCII 值；多字符运算符 (==) 用自定义枚举值

2. 规则结构体：正则匹配规则表

```
static struct rule {
    char *regex;
    int token_type;
} rules[] = {
    {" +", TK_NOTYPE},    // spaces
    {"\\+", '+'},        // plus
    {"==", TK_EQ}        // equal
};
```

- 作用：定义词法规则，是词法分析的依据

3. make_token(): 词法分析核心函数

- 函数功能：将输入字符串 → Token 序列（词法分析）

代码实现

- 修改nemu/src/monitor/debug/expr.c

1. 实现token的定义与声明

```
enum {
    TK_NOTYPE = 256,
    TK_NUMBER,
    TK_HEX,
    TK_REG,
    TK_EQ,
    TK_NEQ,
    TK_AND,
    TK_OR,
    TK_NEGATIVE,
    TK_DEREF
};
```

```
static struct rule {
    char *regex;
    int token_type;
} rules[] = {
    {" +", TK_NOTYPE},
    {"0x[1-9A-Fa-f][0-9A-Fa-f]*", TK_HEX},
    {"[0-9][0-9]*", TK_NUMBER},
    {"\\$(eax|ecx|edx|ebx|esp|ebp|esi|edi|eip|ax|cx|dx|bx|sp|bp|si|di|al|cl|dl|bl|ah|ch|dh|bh)",
    {"==", TK_EQ},
    {"!=", TK_NEQ},
    {"&&", TK_AND},
    {"\\|\\|", TK_OR},
    {"!", '!'},
    {"\\+", '+'},
    {"-", '-'},
    {"\\*", '*'},
    {"\\/", '/'},
    {"\\(", '('},
    {"\\)", ')' }
};
```

2. 在make_token 函数中进行处理

```

static bool make_token(char *e) {
    int pos = 0;
    regmatch_t pm;
    nr_token = 0;
    while (e[pos] != '\0') {
        int i;
        for (i = 0; i < NR_REGEX; i++) {
            if (regexexec(&re[i], e + pos, 1, &pm, 0) == 0 && pm.rm_so == 0) {
                int len = pm.rm_eo;
                if (rules[i].token_type == TK_NOTYPE) { pos += len; break; }
                tokens[nr_token].type = rules[i].token_type;
                strncpy(tokens[nr_token].str, e + pos, len);
                tokens[nr_token].str[len] = '\0';
                nr_token++;
                pos += len;
                break;
            }
        }
        if (i == NR_REGEX) return false;
    }
    return true;
}

```

功能：

词法分析入口函数，将输入的表达式字符串逐字符扫描、正则匹配，切割为标准化 Token 序列，完成字符串→语义单位的转换。

核心思路：

1. **逐字符扫描：**从表达式起始位置开始，依次匹配所有正则规则；
2. **最长匹配原则：**匹配到有效子串后，直接跳过该段字符，继续扫描剩余内容；
3. **特殊处理：**
 - 空格：直接跳过，不影响 Token 序列
 - 寄存器去除开头 \$ ，仅保留寄存器名称（如 \$eax→eax）
 - 严格校验 Token 长度，防止字符串溢出
4. **错误处理：**无匹配规则时，打印错误位置并返回失败，标识非法表达式。

实现算数表达式递归求值、带有负数的算术表达式求值、更复杂的表达式求值

代码实现

括号匹配检查：check_parentheses()


```
static bool check_parentheses(int p, int q) {
    if (tokens[p].type != '(' || tokens[q].type != ')') return false;
    int cnt = 0;
    for (int i = p; i <= q; i++) {
        if (tokens[i].type == '(') cnt++;
        if (tokens[i].type == ')') cnt--;
        if (cnt < 0) return false;
    }
    return cnt == 0;
}
```

功能：

判断 Token 区间[p,q]是否被一对最外层、完全匹配的括号包裹。

核心思路：

1. **采用计数器模拟栈：**左括号+1，右括号-1；
2. 遍历结束计数器为 0，且过程中不为负，匹配；
3. 是递归下降算法处理括号的核心判断条件，决定是否剥离括号。

运算符优先级：get_pri()

```
static int get_pri(int op) {
    switch (op) {
        case TK_DEREF: case TK_NEGATIVE: case '!': return 4; /* 单目（最高）*/
        case '*': case '/': return 3; /* 乘除 */
        case '+': case '-': return 2; /* 加减 */
        case TK_EQ: case TK_NEQ: return 1; /* 比较 */
        case TK_AND: return 0; case TK_OR: return -1; /* 逻辑（最低）*/
        default: return -2;
    }
}
```

功能：

定义所有运算符的优先级数值，数值越大优先级越高，决定运算执行顺序。

核心思路：

1. **严格遵循C 语言运算符优先级标准：**单目运算符 > 算术乘除 > 算术加减 > 比较运算 > 逻辑运算；
2. 单目运算符（负号、解引用、非）优先级最高，保证优先计算；
3. 优先级数值为整数，便于比较大小，实现主运算符查找。

主运算符查找：find_dominant_op()

```
static int find_dominant_op(int p, int q) {
    int cnt = 0;
    int min_pri = 100, pos = -1;
    for (int i = p; i <= q; i++) {
        if (tokens[i].type == '(') cnt++;
        if (tokens[i].type == ')') cnt--;
        if (cnt != 0) continue;
        int t = tokens[i].type;
        if (t == TK_NEGATIVE || t == TK_DEREF || t == '!') continue;
        int pri = get_pri(t);
        if (pri < 0) continue;
        if (pri <= min_pri) {
            min_pri = pri;
            pos = i;
        }
    }
    if (pos == -1) {
        for (int i = p; i <= q; i++) {
            if (tokens[i].type == '(') cnt++;
            if (tokens[i].type == ')') cnt--;
            if (cnt != 0) continue;
            if (tokens[i].type == TK_NEGATIVE || tokens[i].type == TK_DEREF || tokens[i].type == '!')
                return i;
        }
    }
    return pos;
}
```

功能：

递归下降算法核心：在 Token 区间[p,q]中，查找最外层、优先级最低的运算符（主运算符）。

核心思路：

1. **跳过括号内部：**括号内的运算符优先级更高，不参与当前层主运算符查找；
2. **最低优先级优先：**主运算符是最后执行的运算符，对应优先级最低；
3. **左结合性：**相同优先级时，取右侧运算符，符合 C 语言左结合规则；
4. 找到主运算符后，表达式可分裂为左、右两个子表达式，实现递归求解。

表达式求值核心函数：eval()

```

static uint32_t eval(int p, int q, bool *success) {
    if (!*success || p > q) { *success = false; return 0; }
    if (p == q) {
        switch (tokens[p].type) {
            case TK_NUMBER: return atoi(tokens[p].str);
            case TK_HEX: { uint32_t v; sscanf(tokens[p].str, "%x", &v); return v; }
            case TK_REG: {
                if (!strcmp(tokens[p].str, "$eip")) return cpu.eip;
                if (!strcmp(tokens[p].str, "$eax")) return reg_l(0);
                if (!strcmp(tokens[p].str, "$ecx")) return reg_l(1);
                if (!strcmp(tokens[p].str, "$edx")) return reg_l(2);
                if (!strcmp(tokens[p].str, "$ebx")) return reg_l(3);
                if (!strcmp(tokens[p].str, "$esp")) return reg_l(4);
                if (!strcmp(tokens[p].str, "$ebp")) return reg_l(5);
                if (!strcmp(tokens[p].str, "$esi")) return reg_l(6);
                if (!strcmp(tokens[p].str, "$edi")) return reg_l(7);
                *success = false; return 0;
            }
            default: *success = false; return 0;
        }
    }
    if (check_parentheses(p, q)) return eval(p+1, q-1, success);
    int op = find_dominant_op(p, q);
    if (op == -1) { *success = false; return 0; }

    if (tokens[op].type == TK_NEGATIVE) return 0 - eval(op+1, q, success);
    if (tokens[op].type == TK_DEREF) {
        uint32_t addr = eval(op+1, q, success);
        return vaddr_read(addr, 4);
    }
    if (tokens[op].type == '!') return !eval(op+1, q, success);

    uint32_t l = eval(p, op-1, success);
    uint32_t r = eval(op+1, q, success);
    switch(tokens[op].type) {
        case '+': return l + r;
        case '-': return l - r;
        case '*': return l * r;
        case '/': return l / r;
        case TK_EQ: return l == r;
        case TK_NEQ: return l != r;
        case TK_AND: return l && r;
        case TK_OR: return l || r;
        default: *success = false; return 0;
    }
}

```

```
}  
}
```

功能：

递归下降求值主函数，根据 Token 序列的语法结构，分层次、分情况完成表达式计算，是语法分析 + 语义计算的结合体。

核心思路：

采用分治思想，将复杂表达式拆解为简单子表达式，递归求解后合并结果，共分 5 层处理：

1. **原子表达式**：**单个 Token（数字 / 寄存器），直接返回数值；
2. **括号表达式**：**剥离最外层括号，递归求解内部表达式；
3. **查找主运算符**：**确定当前表达式的分裂点；
4. **单目运算符**：**仅需计算右子表达式，执行负号 / 解引用 / 逻辑非；
5. **双目运算符**：**分裂为左、右子表达式，递归求解后执行运算。

模块对外接口：expr()

```
uint32_t expr(char *e, bool *success) {  
    *success = make_token(e);  
    if (!*success) return 0;  
    for (int i = 0; i < nr_token; i++) {  
        if (tokens[i].type == '-') {  
            int prev_type = (i == 0) ? '(' : tokens[i-1].type;  
            if (prev_type == '(' || get_pri(prev_type) >= 0) {  
                tokens[i].type = TK_NEGATIVE;  
            }  
        }  
        if (tokens[i].type == '*') {  
            int prev_type = (i == 0) ? '(' : tokens[i-1].type;  
            if (prev_type == '(' || get_pri(prev_type) >= 0) {  
                tokens[i].type = TK_DEREF;  
            }  
        }  
    }  
    return eval(0, nr_token - 1, success);  
}
```

```

static int cmd_p(char *args) {
    if (!args) { printf("error in expr()\n"); return 0; }
    bool ok;
    uint32_t res = expr(args, &ok);
    if (ok) {
        printf("the value of expr is: 0x%08x\n", res);
    } else {
        printf("error in expr()\n");
    }
    return 0;
}

```

```

static int cmd_x(char *args) {
    if (!args) return 0;
    char *space = strchr(args, ' ');
    if (!space) return 0;
    *space = '\0';
    int n = atoi(args);
    bool ok;
    uint32_t addr = expr(space+1, &ok);
    if (ok) {
        for(int i=0; i<n; i++) {
            printf("0x%08x: 0x%08x\n", addr, vaddr_read(addr,4));
            addr += 4;
        }
    }
    return 0;
}

```

测试结果

- 算术表达式

```
[src/monitor/monitor.c,30,welcome] Build time: 20:38:46, Apr 15 2026
For help, type "help"
(nemu) p --1
the value of expr is: 0x00000001
(nemu) p 1+-1
the value of expr is: 0x00000000
(nemu) p *$eip
the value of expr is: 0x001234b8
(nemu) p *0x100000
the value of expr is: 0x001234b8
(nemu) p *0x100001
the value of expr is: 0x00001234
(nemu) p 1+2*3
the value of expr is: 0x00000007
(nemu) p 1 + 0x00000012
the value of expr is: 0x00000013
```

- 内存扫描

```
(nemu) x 10 $eip
Memory:
0x00100000: 0x001234b8
0x00100004: 0x0027b900
0x00100008: 0x01890010
0x0010000c: 0x0441c766
0x00100010: 0x02bb0001
0x00100014: 0x66000000
0x00100018: 0x009984c7
0x0010001c: 0x01ffffe0
0x00100020: 0x0000b800
0x00100024: 0x00d60000
```

阶段三

实现监视点池的管理

- 修改nemu/include/monitor/watchpoint.h

```

#ifndef __WATCHPOINT_H__
#define __WATCHPOINT_H__

#include "common.h"

// 监视点结构体
typedef struct watchpoint {
    int NO;                // 监视点编号
    struct watchpoint *next; // 链表指针

    char expr[128];        // 监视的表达式
    uint32_t old_val;       // 表达式上一次的值
    int hit;               // 触发次数
} WP;

// 监视池初始化
void init_wp_pool(void);
// 新建监视点
WP* new_wp(char *expr_str);
// 释放监视点
void free_wp(int wp_no);
// 显示所有监视点
void display_wp(void);
// 检查监视点值是否变化
bool check_wp(void);

#endif

```

功能：

定义监视点的核心数据结构，存储监控所需的全部信息；

设计思路：

通过链表指针实现节点串联，配合编号、表达式、旧值、命中计数，完成值变化监控与状态记录。

- 修改 nemu/src/monitor/debug/watchpoint.c

1. 全局变量定义

```

#define NR_WP 32
static WP wp_pool[NR_WP]; // 静态数组，监视点内存池
static WP *head = NULL;   // 已使用监视点链表头
static WP *free_ = NULL;  // 空闲监视点链表头

```

核心设计：池式结构 + 双链表

用固定大小静态数组作为内存池，避免动态内存分配（malloc/free），更稳定高效；

拆分为空闲链表和已使用链表：空闲链表存储未分配的监视点，已使用链表存储正在工作的监视点；节点仅在两个链表间迁移，无内存申请 / 释放，效率极高。

2. `init_wp_pool` 监视池初始化函数

```
void init_wp_pool() {
    int i;
    for (i = 0; i < NR_WP; i++) {
        wp_pool[i].NO = i;
        wp_pool[i].next = &wp_pool[i + 1];
        memset(wp_pool[i].expr, 0, sizeof(wp_pool[i].expr));
        wp_pool[i].old_val = 0;
        wp_pool[i].hit = 0;
    }
    wp_pool[NR_WP - 1].next = NULL;

    head = NULL;
    free_ = wp_pool;
}
```

功能：

对监视点静态内存池、链表结构、全局编号计数器进行统一初始化，是模块的入口初始化函数。构建初始空闲链表，重置所有监视点成员变量，保证模块上电初始状态合法。

核心思路：

- **链表构建：**遍历静态数组，将所有监视点节点串联为空闲单向链表，尾节点next置空，防止遍历越界；
- **成员初始化：**清空表达式字符串缓冲区，初始化旧值、命中次数为默认值；
- **指针初始化：**free_指向空闲链表头，head置空（无激活监视点）；
- **编号初始化：**显式初始化全局编号计数器wp_id=0（规范工程实践，弥补全局变量隐式初始化的缺陷）。

2. `new_wp` 创建监视点函数


```

WP* new_wp(char *expr_str) {
    if (free_ == NULL) { printf("No free watchpoint!\n"); return NULL; }
    WP *p = free_;
    free_ = free_>next;
    p->NO = wp_id++;
    strncpy(p->expr, expr_str, sizeof(p->expr)-1);
    bool success;
    p->old_val = expr(p->expr, &success);
    p->hit = 0;

    p->next = head;
    head = p;
    return p;
}

```

功能：

从空闲链表申请一个监视点节点，绑定用户输入的表达式，计算表达式初始值，将节点加入激活链表，并分配全局唯一编号。

核心设计：

- **资源分配：**从空闲链表头部取节点，时间复杂度 $O(1)$ ，资源分配高效；
- **唯一标识：**使用自增计数器分配编号，删除后不复用，符合调试工具使用习惯；
- **安全拷贝：**使用strncpy限制表达式长度，避免缓冲区溢出；
- **基准值计算：**调用表达式求值模块计算初始值，作为后续对比基准；
- **链表插入：**头插法加入激活链表，提升节点插入效率。

3. free_wp 删除监视点函数

```

void free_wp(int wp_no) {
    WP **pp = &head;
    while (*pp) {
        if ((*pp)->NO == wp_no) {
            WP *p = *pp;
            *pp = p->next;
            p->next = free_;
            free_ = p;
            printf("Delete watchpoint %d\n", wp_no);
            return;
        }
        pp = &(*pp)->next;
    }
    printf("Watchpoint %d not found\n", wp_no);
}

```

功能：

根据监视点编号，从激活链表中摘除目标节点，重置节点状态并回收至空闲链表，实现监视点资源复用。

核心设计：

- ****二级指针遍历：****通过二级指针遍历：无需区分头节点 / 普通节点，简化链表删除逻辑，避免空指针异常；
- ****节点摘除：****修改前驱节点指针，将目标节点从激活链表移除；
- ****资源回收：****将节点插入空闲链表头部，完成资源复用；
- ****异常处理：****未找到目标节点时输出提示，提升模块健壮性

问题：static的使用

static 定义全局静态变量，该变量只能在本文件中进行访问，保护了数据的安全性。其他源文件想对该变量进行操作时，需要通过封装的函数来实现，避免直接访问全局变量导致的程序错误。

实现监视点

- 修改 nemu/src/monitor/debug/watchpoint.c

1. display_wp 显示监视点函数

```
void display_wp(void) {
    if (head == NULL) { printf("No watchpoints.\n"); return; }
    printf("NO\tEXPR\t\tHIT\n");
    WP *p = head;
    while (p) {
        printf("%d\t%s\t%d\n", p->NO, p->expr, p->hit);
        p = p->next;
    }
}
```

功能：

遍历激活链表，格式化输出所有监视点的编号、表达式、命中次数，为用户提供可视化调试界面。

核心设计：

- **空值判断：**无激活监视点时直接返回，避免无效遍历；
- **线性遍历：**顺序遍历激活链表，只读访问节点数据；
- **格式化输出：**表格化打印信息，提升交互可读性；
- **无锁设计：**仅读取数据，不修改链表结构，保证执行安全。

2. check_wp 监视点检测函数

```

bool check_wp(void) {
    WP *p = head;
    bool hit = false;
    while (p) {
        bool success;
        uint32_t new_val = expr(p->expr, &success);
        if (success && new_val != p->old_val) {
            hit = true;
            p->hit ++;
            printf("\nWatchpoint %d hit!\n", p->NO);
            printf("Expr: %s\n", p->expr);
            printf("Old: 0x%08x, New: 0x%08x\n", p->old_val, new_val);
            p->old_val = new_val;
        }
        p = p->next;
    }
    return !hit;
}

```

功能：

模拟器每条指令执行完成后调用，实时计算所有监视点表达式的当前值，对比新旧值判断是否触发监视点；触发时更新状态并暂停模拟器。

核心设计：

- **实时求值：**
- 调用表达式求值**调用表达式模块获取最新值，保证监控精度；
- **值对比逻辑：**仅当计算成功且值发生变化时触发监视点；
- **状态更新：**触发后自增命中次数，打印新旧值并更新基准值；
- **执行控制：**返回值控制模拟器流程 —— 触发返回false(暂停)，未触发返回true(继续)；
- **容错机制：**表达式非法时自动跳过，不影响模拟器运行。

-
- 修改 nemu/src/monitor/debug/ui.c

1. 监视点申请和删除

```
static int cmd_w(char *args) {
    if (!args) {
        printf("Usage: w <expression>\n");
        return 0;
    }
    new_wp(args);
    return 0;
}
static int cmd_d(char *args) {
    if (!args) {
        printf("Usage: d <watchpoint_number>\n");
        return 0;
    }
    int num = atoi(args);
    free_wp(num);
    return 0;
}
```

2. 监视点信息查看（在cmd_info中补充）

```
if(s == 'w'){
    display_wp();
    return 0;
}
```

- 修改 nemu/src/monitor/cpu-exec.c

1. 添加头文件

```
#include "monitor/watchpoint.h"
```

2. 添加cpu_exec()函数

```
#ifdef DEBUG
    if(check_wp() == false){
        nemu_state = NEMU_STOP;
    }
#endif
```

测试

步骤：

1. 输入 `p 1+2` 测试表达式求值
2. 输入 `w $eax` 申请监视点
3. 输入 `info w` 查看监视点信息
4. 输入 `si` 单步执行
5. 输入 `d 0` 删除监视点
6. 输入 `info w` 查看监视点信息
7. 输入 `q` 退出模拟器

```
[src/monitor/monitor.c,47,load_default_img] No
image is given. Use the default build-in imag
e.
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time:
22:15:26, Apr 15 2026
For help, type "help"
(nemu) p 1+2
the value of expr is: 0x00000003
(nemu) w $eax
Set watchpoint 0: $eax
(nemu) info w
NO      EXPR      HIT
0       $eax      0
(nemu) si
100000:  b8 34 12 00 00
        movl $0x1234,%eax

Watchpoint 0 hit!
Expr: $eax
Old: 0x2cd6b437, New: 0x00001234
(nemu) d 0
Delete watchpoint 0
(nemu) info w
No watchpoints.
(nemu) q
```

必答题

1. 查i386手册

- EFLAGS寄存器的CF位是什么意思

2-8 EFLAGS Register

4-1 Systems Flags of EFLAGS Register

- ModR/M字节是什么

17.2 INSTRUCTION FORMAT

17.2.1 ModR/M and SIB Bytes

- mov指令的格式是什么

17.2 INSTRUCTION FORMAT

17.2.2.11 Instruction Set Detail

MOV — Move Data.

MOV — Move to/from Special Registers

2. shell命令

- 统计 nemu / 目录下所有.c 和.h 文件总行数

```
find nemu -name "*.c" -o -name "*.h" | xargs wc -l
```

```

liuyuze@liuyuze-virtual-machine:~/ics2017$ find nemu -name "*.c" -o -name "*.h" | xargs wc -l
 45 nemu/include/debug.h
 21 nemu/include/monitor/watchpoint.h
  7 nemu/include/monitor/monitor.h
  8 nemu/include/monitor/expr.h
 13 nemu/include/device/port-io.h
 14 nemu/include/device/mmio.h
 13 nemu/include/macro.h
115 nemu/include/cpu/decode.h
189 nemu/include/cpu/rtl.h
 55 nemu/include/cpu/reg.h
 53 nemu/include/cpu/exec.h
 82 nemu/include/memory/mmu.h
 18 nemu/include/memory/memory.h
 29 nemu/include/common.h
  8 nemu/include/nemu.h
 31 nemu/src/misc/logo.c
150 nemu/src/monitor/debug/ui.c
215 nemu/src/monitor/debug/expr.c
 95 nemu/src/monitor/debug/watchpoint.c
157 nemu/src/monitor/diff-test/diff-test.c
 48 nemu/src/monitor/diff-test/protocol.h
106 nemu/src/monitor/diff-test/gdb-host.c
314 nemu/src/monitor/diff-test/protocol.c
143 nemu/src/monitor/monitor.c
 48 nemu/src/monitor/cpu-exec.c
 28 nemu/src/device/timer.c
 70 nemu/src/device/keyboard.c
 70 nemu/src/device/io/mmio.c
 56 nemu/src/device/io/port-io.c
 98 nemu/src/device/device.c
 38 nemu/src/device/vga.c
 29 nemu/src/device/serial.c
 43 nemu/src/cpu/reg.c
113 nemu/src/cpu/decode/modrm.c
311 nemu/src/cpu/decode/decode.c
 13 nemu/src/cpu/intr.c
223 nemu/src/cpu/exec/arith.c
 46 nemu/src/cpu/exec/special.c
 43 nemu/src/cpu/exec/control.c
 65 nemu/src/cpu/exec/system.c
 60 nemu/src/cpu/exec/logic.c
 77 nemu/src/cpu/exec/data-mov.c
  8 nemu/src/cpu/exec/all-instr.h
  9 nemu/src/cpu/exec/prefix.c
 32 nemu/src/cpu/exec/cc.c
255 nemu/src/cpu/exec/exec.c
 28 nemu/src/memory/memory.c
 12 nemu/src/main.c
3704 总用量

```

- 统计 PA1 编写的代码行数

1. 切回框架代码分支，统计原始行数

```
git checkout master
```

```
find nemu -name "*.c" -o -name "*.h" | xargs wc -l
```

2. 切回pa1分支，统计当前行数

```
git checkout pa1
```

```
find nemu -name "*.c" -o -name "*.h" | xargs wc -l
```

```

liuyuze@liuyuze-virtual-machine:~/ics2017$ find nemu -name "*.c" -o -name "*.h" | xargs wc -l
45 nemu/include/debug.h
15 nemu/include/monitor/watchpoint.h
7 nemu/include/monitor/monitor.h
8 nemu/include/monitor/expr.h
13 nemu/include/device/port-io.h
14 nemu/include/device/mmio.h
13 nemu/include/macro.h
115 nemu/include/cpu/decode.h
189 nemu/include/cpu/rtl.h
60 nemu/include/cpu/reg.h
53 nemu/include/cpu/exec.h
82 nemu/include/memory/mmu.h
18 nemu/include/memory/memory.h
29 nemu/include/common.h
8 nemu/include/nemu.h
31 nemu/src/misc/logo.c
116 nemu/src/monitor/debug/ui.c
109 nemu/src/monitor/debug/expr.c
23 nemu/src/monitor/debug/watchpoint.c
157 nemu/src/monitor/diff-test/diff-test.c
48 nemu/src/monitor/diff-test/protocol.h
106 nemu/src/monitor/diff-test/gdb-host.c
314 nemu/src/monitor/diff-test/protocol.c
143 nemu/src/monitor/monitor.c
44 nemu/src/monitor/cpu-exec.c
28 nemu/src/device/timer.c
70 nemu/src/device/keyboard.c
70 nemu/src/device/io/mmio.c
56 nemu/src/device/io/port-io.c
98 nemu/src/device/device.c
38 nemu/src/device/vga.c
29 nemu/src/device/serial.c
43 nemu/src/cpu/reg.c
113 nemu/src/cpu/decode/modrm.c
311 nemu/src/cpu/decode/decode.c
13 nemu/src/cpu/intr.c
223 nemu/src/cpu/exec/arith.c
46 nemu/src/cpu/exec/special.c
43 nemu/src/cpu/exec/control.c
65 nemu/src/cpu/exec/system.c
60 nemu/src/cpu/exec/logic.c
77 nemu/src/cpu/exec/data-mov.c
8 nemu/src/cpu/exec/all-instr.h
9 nemu/src/cpu/exec/prefix.c
32 nemu/src/cpu/exec/cc.c
255 nemu/src/cpu/exec/exec.c
28 nemu/src/memory/memory.c
12 nemu/src/main.c
3487 总用量

```

pa1 编写的代码行数 = 3704 - 3487 = 217

- 统计去除空行后的总行数

```
find nemu -name "*.c" -o -name "*.h" | xargs grep -v "^$" | wc -l
```

```

liuyuze@liuyuze-virtual-machine:~/ics2017$ git branch
master
pa0
* pa1
liuyuze@liuyuze-virtual-machine:~/ics2017$ find nemu -name "*.c" -o -name "*.h" | xargs grep -v "^$" | wc -l
3049

```

3. GCC 编译

Wall:

- 作用：开启 GCC所有常用编译警告，检测代码中潜在问题（如未使用变量、类型不匹配、函数未声明、忽略返回值等）。
- 特点：仅提示警告，不阻止编译。

Werror:

- 作用：将所有编译警告升级为编译错误，只要存在警告就无法生成可执行文件。

为什么使用这两个选项：

- **Wall**： 开启所有常用编译警告，帮助发现代码中的潜在问题。
- **-Werror**： 将警告升级为错误，强制修复问题，确保代码质量。